

MODULE3

Module 3 – Object-Oriented Python, you need to complete the OOP topics and build the **Python Banking System**,

Class

A **class** is a blueprint or template used to create objects in Python. It defines the **properties (data)** and **methods (functions)** that an object can have.

Syntax:

```
class Student:  
    Pass
```

Object

An **object** is an instance of a class. It is created from a class and can access the data and methods defined inside that class.

EXAMPLE:-

```
class Student:  
    name = "Ammu"  
    age = 19  
student1 = Student()  
print(student1.name)  
print(student1.age)
```

OUTPUT:-

```
Ammu  
19
```

Constructor

A **constructor** is a special method that is automatically called when we create an object.

In Python, the constructor is written as:

```
__init__()
```

Why do we use a constructor?

We use a constructor to **initialize (set) the values of an object** when it is created.

Example:

```
class Student:  
    def __init__(self, name, age):  
        self.name = name
```

```
        self.age = age
student1 = Student("Honey", 19)
print(student1.name)
print(student1.age)
```

Output:

```
Honey
19
```

Methods

A **method** is a function that is defined inside a class. Methods are used to **perform actions or operations** on an object.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
    def display(self):
        print("Student Name:", self.name)
student1 = Student("Honey")
student1.display()
```

Output:

```
Student Name: Honey
```

OOP Concepts

1. Encapsulation

Keeping data and methods together inside a class.

2. Inheritance

One class can use the properties and methods of another class.

3. Polymorphism

The same method name can behave differently in different classes.

4. Abstraction

Showing only the important details and hiding unnecessary implementation

EXAMPLE:-

```
class BankAccount:
```

```

# Constructor
def __init__(self, name, balance):
    self.name = name
    self.__balance = balance
# Method
def deposit(self, amount):
    self.__balance += amount
    print("Amount deposited:", amount)
# Method
def show_balance(self):
    print("Account Holder:", self.name)
    print("Balance:", self.__balance)

# Object
account1 = BankAccount("Ammu", 5000)

account1.show_balance()

account1.deposit(2000)

account1.show_balance()

```

Output:-

```

Account Holder: Ammu
Balance: 5000
Amount deposited: 2000
Account Holder: Ammu
Balance: 7000

```

MINI PROJECT

class BankAccount:

```

class BankAccount:

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount
        print("Money deposited successfully")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance = self.balance - amount
            print("Money withdrawn successfully")
        else:
            print("Insufficient balance")

    def show_balance(self):
        print("Name:", self.name)
        print("Balance:", self.balance)

```

```
# Creating object
account = BankAccount("Bujjiamma", 5000)

account.show_balance()

account.deposit(1000)

account.withdraw(2000)

account.show_balance()
```

Output

```
Name: Bujjiamma
Balance: 5000
Money deposited successfully
Money withdrawn successfully
Name: Bujjiamma
Balance: 4000
```